

Aula 18 - Autenticação e Autorização com Spring Security

Duração: 2 horas

Objetivos:

- Implementar autenticação JWT em Spring Boot
- Configurar controle de acesso baseado em roles
- Testar segurança com chamadas autenticadas

Parte Teórica (30 min)

1. Fluxo JWT

Diagram

Code

```
sequenceDiagram
    Client->>+AuthAPI: Login (username/password)
    AuthAPI-->>-Client: JWT Token
    Client->>+ResourceAPI: Request com JWT
    ResourceAPI-->>-Client: Dados protegidos
```

2. Componentes do Spring Security

Componente	Função
SecurityFilterChain	Define regras de segurança
UserDetailsService	Carrega dados do usuário
JwtAuthenticationFilter	Valida tokens JWT

Atividade Prática (1h30min)

Passo 1: Configuração Inicial

1. Adicione as dependências (pom.xml)

xml

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-security</artifactId>
</dependency>
<dependency>
  <groupId>io.jsonwebtoken</groupId>
  <artifactId>jjwt-api</artifactId>
  <version>0.11.5</version>
</dependency>
```

2. Configure o SecurityFilterChain

java

```
// src/main/java/com/example/projeto/config/SecurityConfig.java
@Configuration
@EnableWebSecurity
public class SecurityConfig {

    @Bean
    public SecurityFilterChain filterChain(HttpSecurity http) throws Exception {
        http
            .csrf().disable()
            .authorizeHttpRequests(auth -> auth
                .requestMatchers("/auth/**").permitAll()
                .requestMatchers("/admin/**").hasRole("ADMIN")
                .anyRequest().authenticated()
            )
            .addFilterBefore(jwtAuthFilter(), UsernamePasswordAuthenticationFilter.class);

        return http.build();
    }
}
```

Passo 2: Implementação JWT

1. Serviço de Geração de Token

java

```
// src/main/java/com/example/projeto/auth/JwtService.java
@Service
public class JwtService {
    private final String SECRET_KEY = "mySecretKey123!";
```

```

public String generateToken(UserDetails user) {
    return Jwts.builder()
        .setSubject(user.getUsername())
        .setIssuedAt(new Date())
        .setExpiration(new Date(System.currentTimeMillis() + 1000 * 60 * 60)) // 1h
        .signWith(SignatureAlgorithm.HS256, SECRET_KEY)
        .compact();
}
}

```

2. Filtro de Autenticação

java

```

public class JwtAuthFilter extends OncePerRequestFilter {

    @Override
    protected void doFilterInternal(HttpServletRequest request,
                                    HttpServletResponse response,
                                    FilterChain chain) throws IOException, ServletException
    {

        String token = extractToken(request);
        if (token != null && jwtService.validateToken(token)) {
            // Configura autenticação no SecurityContext
        }
        chain.doFilter(request, response);
    }
}

```

Passo 3: Testes com Postman

1. Requisição de Login

http

POST /auth/login HTTP/1.1
Content-Type: application/json

```

{
    "username": "admin",
    "password": "123456"
}

```

2. Acesso a Recurso Protegido

http

```
GET /api/produtos HTTP/1.1
Authorization: Bearer <JWT_TOKEN>
```

Checklist de Entrega

- Rota de login gerando JWT
- Endpoint protegido acessível apenas com token
- Testes Postman documentados em `docs/auth-guide.md`

Tarefa Pós-Aula

1. Implemente refresh tokens
2. Adicione controle de roles no frontend

Exemplo de Refresh Token:

java

```
public TokenResponse refreshToken(String refreshToken) {
    if (jwtService.validateToken(refreshToken)) {
        String username = jwtService.extractUsername(refreshToken);
        UserDetails user = userService.loadUserByUsername(username);
        return new TokenResponse(jwtService.generateToken(user));
    }
    throw new InvalidTokenException();
}
```

Próximos Passos

- Aula 19: Deploy no Heroku/AWS
- Aula 20: Apresentação Final

Dica Pro:

"Use `@PreAuthorize` para controle granular de métodos!"

Recursos:

- [JWT Official Docs](#)
- [Spring Security](#)

Sua aplicação agora está segura!  